ASSIGNMENT-12.3

HT NO:2303A51585

Batch No:28

Course :AI Assisted Coding

Task 1: Sorting Student Records for Placement Drive

Scenario

SR University's Training and Placement Cell needs to shortlist candidates efficiently during campus placements. Student records must be sorted by CGPA in descending order.

Tasks

1. Use GitHub Copilot to generate a program that stores student records (Name, Roll Number, CGPA).

2. Implement the following sorting algorithms using AI assistance:

o Quick Sort

o Merge Sort

3. Measure and compare runtime performance for large datasets.

4. Write a function to display the top 10 students based on CGPA.

Prompt:

Generate a Python program to store student records (Name, Roll No, CGPA) and implement Quick Sort and Merge Sort to sort by CGPA descending. Also compare runtime performance for large datasets and display top 10 students.

Python Code:

```python
#Generate a Python program to store student records (Name, Roll No, CGPA) and implement Quick Sort and Merge S
import time
class Student:
    """
    A class to represent a student record.

    Attributes:
        name (str): The name of the student
        roll_no (int): The roll number of the student
        cgpa (float): The CGPA of the student
    """

    def __init__(self, name: str, roll_no: int, cgpa: float):
        """Initialize the student record."""
        self.name = name
        self.roll_no = roll_no
        self.cgpa = cgpa

    def __repr__(self):
        """Return a string representation of the student record."""
        return f"Student(Name: {self.name}, Roll No: {self.roll_no}, CGPA: {self.cgpa})"
def quick_sort(students: list) -> list:
    """Sort the students by CGPA in descending order using Quick Sort."""
    if len(students) <= 1:
        return students
    pivot = students[len(students) // 2].cgpa
    left = [x for x in students if x.cgpa > pivot]
    middle = [x for x in students if x.cgpa == pivot]
    right = [x for x in students if x.cgpa < pivot]
    return quick_sort(left) + middle + quick_sort(right)
def merge_sort(students: list) -> list:
    """Sort the students by CGPA in descending order using Merge Sort."""
    if len(students) <= 1:
        return students
    mid = len(students) // 2
    left = merge_sort(students[:mid])
```

```python
def merge_sort(students: list) -> list:
    right = merge_sort(students[mid:])
    return merge(left, right)
def merge(left: list, right: list) -> list:
    """Merge two lists of students sorted by CGPA in descending order."""
    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i].cgpa > right[j].cgpa:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1
    result.extend(left[i:])
    result.extend(right[j:])
    return result
# Example usage:
if __name__ == "__main__":
    # Create a list of student records
    students = [
        Student("Alice", 1, 3.8),
        Student("Bob", 2, 3.6),
        Student("Charlie", 3, 3.9),
        Student("David", 4, 3.7),
        Student("Eve", 5, 3.5),
        Student("Frank", 6, 3.4),
        Student("Grace", 7, 3.9),
        Student("Heidi", 8, 3.2),
        Student("Ivan", 9, 3.1),
        Student("Judy", 10, 3.0)
    ]

    # Measure runtime performance for Quick Sort
    start_time = time.time()
```

```python
        Student("Eve", 5, 3.5),
        Student("Frank", 6, 3.4),
        Student("Grace", 7, 3.9),
        Student("Heidi", 8, 3.2),
        Student("Ivan", 9, 3.1),
        Student("Judy", 10, 3.0)
    ]

    # Measure runtime performance for Quick Sort
    start_time = time.time()
    sorted_students_quick = quick_sort(students)
    end_time = time.time()
    print(f"Quick Sort Time: {end_time - start_time:.6f} seconds")

    # Measure runtime performance for Merge Sort
    start_time = time.time()
    sorted_students_merge = merge_sort(students)
    end_time = time.time()
    print(f"Merge Sort Time: {end_time - start_time:.6f} seconds")

    # Display top 10 students sorted by CGPA
    print("\nTop 10 Students (sorted by CGPA):")
    for student in sorted_students_quick[:10]:
        print(student)
```

Output:

```
Quick Sort Time: 0.000040 seconds
Merge Sort Time: 0.000063 seconds

Top 10 Students (sorted by CGPA):
Student(Name: Charlie, Roll No: 3, CGPA: 3.9)
Student(Name: Grace, Roll No: 7, CGPA: 3.9)
Student(Name: Alice, Roll No: 1, CGPA: 3.8)
Student(Name: Alice, Roll No: 1, CGPA: 3.8)
Student(Name: David, Roll No: 4, CGPA: 3.7)
Student(Name: Bob, Roll No: 2, CGPA: 3.6)
Student(Name: Eve, Roll No: 5, CGPA: 3.5)
Student(Name: Frank, Roll No: 6, CGPA: 3.4)
Student(Name: Heidi, Roll No: 8, CGPA: 3.2)
Student(Name: Ivan, Roll No: 9, CGPA: 3.1)
Student(Name: Judy, Roll No: 10, CGPA: 3.0)
(.venv) PS C:\Users\DELL\OneDrive\Desktop\LEARNING COURSES\AI_Assisant_Coding>
```

Justification:

1.Required to efficiently shortlist students based on CGPA.

2.Quick Sort gives fast average performance O(n log n).

3.Merge Sort ensures stable and consistent O(n log n) performance.

4.Performance comparison helps understand real-world efficiency.

5.Top 10 display simplifies recruiter decision-making.

Task 2: Implementing Bubble Sort with AI Comments

• **Task:** Write a Python implementation of Bubble Sort.

• Instructions:

• Students implement Bubble Sort normally.

• Ask AI to generate inline comments explaining key logic (like

swapping, passes, and termination).

• Request AI to provide time complexity analysis.

Prompt:

Add inline comments to a Bubble Sort implementation explaining swapping, passes, and termination. Also provide time complexity.

Python Code:

```python
#Add inline comments to a Bubble Sort implementation explaining swapping, passes, and term
def bubble_sort(arr):
    """
    Sorts an array using the Bubble Sort algorithm.

    Args:
        arr: A list of elements to be sorted

    Returns:
        A sorted list of elements
    """
    n = len(arr)

    # Traverse through all elements in the array
    for i in range(n):
        # Initialize a flag to check if any swapping occurs
        swapped = False

        # Last i elements are already in place, no need to check them
        for j in range(0, n-i-1):
            # Compare adjacent elements
            if arr[j] > arr[j+1]:
                # Swap if the element found is greater than the next element
                arr[j], arr[j+1] = arr[j+1], arr[j]
                swapped = True

        # If no swapping occurred, the array is already sorted
        if not swapped:
            break

    return arr
# Example usage:
if __name__ == "__main__":
    unsorted_array = [64, 34, 25, 12, 22, 11, 90]
    print("Unsorted array:", unsorted_array)
```

```python
def bubble_sort(arr):
            # Compare adjacent elements
            if arr[j] > arr[j+1]:
                # Swap if the element found is greater than the next element
                arr[j], arr[j+1] = arr[j+1], arr[j]
                swapped = True

        # If no swapping occurred, the array is already sorted
        if not swapped:
            break

    return arr
# Example usage:
if __name__ == "__main__":
    unsorted_array = [64, 34, 25, 12, 22, 11, 90]
    print("Unsorted array:", unsorted_array)

    sorted_array = bubble_sort(unsorted_array)
    print("Sorted array:", sorted_array)
```

Output:

```
Unsorted array: [64, 34, 25, 12, 22, 11, 90]
Sorted array: [11, 12, 22, 25, 34, 64, 90]
(.venv) PS C:\Users\DELL\OneDrive\Desktop\LEARNING COURSES\AI_Assisant_Coding>
```

Justification:

1. Helps understand basic sorting logic (comparison & swapping).
2. Easy to implement and suitable for beginners.
3. AI comments improve clarity and readability.
4. Best case O(n), average/worst case $O(n^2)$.

5. Suitable only for small datasets and learning purposes

Task 3: Quick Sort and Merge Sort Comparison

• **Task:** Implement Quick Sort and Merge Sort using recursion.

• Instructions:

• Provide AI with partially completed functions for recursion.

• Ask AI to complete the missing logic and add docstrings.

• Compare both algorithms on random, sorted, and reverse-sorted lists.

Prompt:

Complete recursive Quick Sort and Merge Sort functions with docstrings and compare them on random, sorted, and reverse-sorted lists.

Python Code:

```
#Complete recursive Quick Sort and Merge Sort functions with docstrings and compare th
import random
def quick_sort(arr):
    """
    Sorts an array using the Quick Sort algorithm.

    Args:
        arr: A list of elements to be sorted

    Returns:
        A new list containing the sorted elements
    """
    if len(arr) <= 1:
        return arr
    pivot = arr[len(arr) // 2]
    left = [x for x in arr if x < pivot]
    middle = [x for x in arr if x == pivot]
    right = [x for x in arr if x > pivot]
    return quick_sort(left) + middle + quick_sort(right)
def merge_sort(arr):
    """
    Sorts an array using the Merge Sort algorithm.

    Args:
        arr: A list of elements to be sorted
    Returns:
        A new list containing the sorted elements
    """
    if len(arr) <= 1:
        return arr
    mid = len(arr) // 2
    left_half = merge_sort(arr[:mid])
    right_half = merge_sort(arr[mid:])
    return merge(left_half, right_half)
def merge(left, right):
    """
```

```python
def merge(left, right):
    """
    Merges two sorted lists into a single sorted list.
    Args:
        left: A sorted list
        right: Another sorted list
    Returns:
        A new list containing all elements from left and right, sorted
    """
    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i] < right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1
    result.extend(left[i:])
    result.extend(right[j:])
    return result
# Example usage and comparison
if __name__ == "__main__":
    # Generate random, sorted, and reverse-sorted lists
    random_list = [random.randint(1, 100) for _ in range(10)]
    sorted_list = sorted(random_list)
    reverse_sorted_list = sorted(random_list, reverse=True)

    print("Random List:", random_list)
    print("Sorted List:", sorted_list)
    print("Reverse Sorted List:", reverse_sorted_list)

    # Test Quick Sort
    print("\nQuick Sort Results:")
    print("Random List:", quick_sort(random_list))
    print("Sorted List:", quick_sort(sorted_list))
```

```python
if __name__ == "__main__":
    # Generate random, sorted, and reverse-sorted lists
    random_list = [random.randint(1, 100) for _ in range(10)]
    sorted_list = sorted(random_list)
    reverse_sorted_list = sorted(random_list, reverse=True)

    print("Random List:", random_list)
    print("Sorted List:", sorted_list)
    print("Reverse Sorted List:", reverse_sorted_list)

    # Test Quick Sort
    print("\nQuick Sort Results:")
    print("Random List:", quick_sort(random_list))
    print("Sorted List:", quick_sort(sorted_list))
    print("Reverse Sorted List:", quick_sort(reverse_sorted_list))

    # Test Merge Sort
    print("\nMerge Sort Results:")
    print("Random List:", merge_sort(random_list))
    print("Sorted List:", merge_sort(sorted_list))
    print("Reverse Sorted List:", merge_sort(reverse_sorted_list))
```

Output:

```
ipts\python.exe" "c:/Users/DELL/OneDrive/Desktop/LEARNING COURSES/AI_Assisant_Coding/program-1
Random List: [75, 18, 98, 11, 13, 100, 64, 84, 65, 78]
Sorted List: [11, 13, 18, 64, 65, 75, 78, 84, 98, 100]
Reverse Sorted List: [100, 98, 84, 78, 75, 65, 64, 18, 13, 11]

Quick Sort Results:
Random List: [11, 13, 18, 64, 65, 75, 78, 84, 98, 100]
Sorted List: [11, 13, 18, 64, 65, 75, 78, 84, 98, 100]
Reverse Sorted List: [11, 13, 18, 64, 65, 75, 78, 84, 98, 100]

Merge Sort Results:
Random List: [11, 13, 18, 64, 65, 75, 78, 84, 98, 100]
Sorted List: [11, 13, 18, 64, 65, 75, 78, 84, 98, 100]
Reverse Sorted List: [11, 13, 18, 64, 65, 75, 78, 84, 98, 100]
(.venv) PS C:\Users\DELL\OneDrive\Desktop\LEARNING COURSES\AI_Assisant_Coding>
```

Justification:

1.Demonstrates divide-and-conquer approach.

2.Quick Sort fast on average but worst case $O(n^2)$.

3. Merge Sort guarantees O(n log n) in all cases.

4. Comparison on different inputs shows behavior differences.
5. Builds understanding of algorithm efficiency analysis.

Task 4 :Real-Time Application Inventory Management System

Scenario: A retail store's inventory system contains thousands of products,

each with attributes like product ID, name, price, and stock quantity. Store staff

need to:

1. Quickly search for a product by ID or name.

2. Sort products by price or quantity for stock analysis.

Task:

• Use AI to suggest the most efficient search and sort algorithms for this

use case.

• Implement the recommended algorithms in Python.

• Justify the choice based on dataset size, update frequency, and

performance requirements.

Prompt:

Generate a simple Python Inventory Management System program.
Each product should have product_id, name, price, and stock_quantity.
Use a dictionary for fast search by product ID.
Implement search by name.
Implement Merge Sort to sort by price.
Implement Quick Sort to sort by stock quantity.
Add comments and time complexity explanation

Python Code:

```python
class Product:
    """
    Represents a single product in the inventory.

    Attributes:
        product_id (int): Unique product ID
        name (str): Product name
        price (float): Product price
        stock_quantity (int): Available stock
    """

    def __init__(self, product_id: int, name: str, price: float, stock_quantity: int):
        self.product_id = product_id
        self.name = name
        self.price = price
        self.stock_quantity = stock_quantity

    def __repr__(self):
        """String representation of product"""
        return f"{self.product_id} | {self.name} | ₹{self.price} | Stock: {self.stock_quantity}"


class InventoryManagementSystem:
    """
    Inventory system using dictionary for fast product lookup.

    Data Structure Used:
    - Dictionary → O(1) average time complexity for search by product_id
    """

    def __init__(self):
        self.products = {}  # {product_id: Product}

    # ----------------------------------------------------------
    # ADD PRODUCT
    # Time Complexity: O(1)
```

```python
class InventoryManagementSystem:
    # ADD PRODUCT
    # Time Complexity: O(1)
    # -------------------------------------------------------
    def add_product(self, product: Product):
        self.products[product.product_id] = product

    # -------------------------------------------------------
    # SEARCH BY PRODUCT ID
    # Time Complexity: O(1)
    # -------------------------------------------------------
    def search_by_id(self, product_id: int):
        return self.products.get(product_id, None)

    # -------------------------------------------------------
    # SEARCH BY NAME
    # Time Complexity: O(n)
    # -------------------------------------------------------
    def search_by_name(self, query: str):
        return [
            product for product in self.products.values()
            if query.lower() in product.name.lower()
        ]

    # -------------------------------------------------------
    # MERGE SORT BY PRICE
    # Time Complexity: O(n log n)
    # Space Complexity: O(n)
    # -------------------------------------------------------
    def merge_sort_by_price(self):

        def merge_sort(product_list):
            if len(product_list) <= 1:
                return product_list

            mid = len(product_list) // 2
```

```python
        mid = len(product_list) // 2
        left = merge_sort(product_list[:mid])
        right = merge_sort(product_list[mid:])

        return merge(left, right)

    def merge(left, right):
        sorted_list = []
        i = j = 0

        while i < len(left) and j < len(right):
            if left[i].price < right[j].price:
                sorted_list.append(left[i])
                i += 1
            else:
                sorted_list.append(right[j])
                j += 1

        sorted_list.extend(left[i:])
        sorted_list.extend(right[j:])

        return sorted_list

    return merge_sort(list(self.products.values()))

# ---------------------------------------------------------
# QUICK SORT BY STOCK QUANTITY
# Average Time Complexity: O(n log n)
# Worst Case: O(n²)
# Space Complexity: O(log n)
# ---------------------------------------------------------
def quick_sort_by_stock(self):
```

```python
    def quick_sort_by_stock(self):

        def quick_sort(product_list):
            if len(product_list) <= 1:
                return product_list

            pivot = product_list[0]

            less = [p for p in product_list if p.stock_quantity < pivot.stock_quantity]
            equal = [p for p in product_list if p.stock_quantity == pivot.stock_quantity]
            greater = [p for p in product_list if p.stock_quantity > pivot.stock_quantity]

            return quick_sort(less) + equal + quick_sort(greater)

        return quick_sort(list(self.products.values()))


# ---------------------------------------------------------
# MAIN PROGRAM
# ---------------------------------------------------------
if __name__ == "__main__":

    inventory = InventoryManagementSystem()

    # Adding products (O(1))
    inventory.add_product(Product(1, "Smartphone", 699.99, 50))
    inventory.add_product(Product(2, "Laptop", 999.99, 30))
    inventory.add_product(Product(3, "Headphones", 199.99, 100))
    inventory.add_product(Product(4, "Smartwatch", 299.99, 20))

    # Search by Name (O(n))
    print("\nSearch Results for 'smart':")
    results = inventory.search_by_name("smart")
    for p in results:
```

```python
    inventory = InventoryManagementSystem()

    # Adding products (O(1))
    inventory.add_product(Product(1, "Smartphone", 699.99, 50))
    inventory.add_product(Product(2, "Laptop", 999.99, 30))
    inventory.add_product(Product(3, "Headphones", 199.99, 100))
    inventory.add_product(Product(4, "Smartwatch", 299.99, 20))

    # Search by Name (O(n))
    print("\nSearch Results for 'smart':")
    results = inventory.search_by_name("smart")
    for p in results:
        print(p)

    # Sort by Price using Merge Sort
    print("\nProducts Sorted by Price (Merge Sort):")
    sorted_by_price = inventory.merge_sort_by_price()
    for p in sorted_by_price:
        print(p)

    # Sort by Stock using Quick Sort
    print("\nProducts Sorted by Stock (Quick Sort):")
    sorted_by_stock = inventory.quick_sort_by_stock()
    for p in sorted_by_stock:
        print(p)
```

Output:

```
Search Results for 'smart':
1 | Smartphone | ₹699.99 | Stock: 50
4 | Smartwatch | ₹299.99 | Stock: 20

Products Sorted by Price (Merge Sort):
3 | Headphones | ₹199.99 | Stock: 100
4 | Smartwatch | ₹299.99 | Stock: 20
1 | Smartphone | ₹699.99 | Stock: 50
2 | Laptop | ₹999.99 | Stock: 30

Products Sorted by Stock (Quick Sort):
4 | Smartwatch | ₹299.99 | Stock: 20
2 | Laptop | ₹999.99 | Stock: 30
1 | Smartphone | ₹699.99 | Stock: 50
3 | Headphones | ₹199.99 | Stock: 100
(.venv) PS C:\Users\DELL\OneDrive\Desktop\LEARNING COURSES\AI
```

Justification:

1. Hash Map enables O(1) fast product ID search.

2. Efficient searching is critical for large inventories.

3. Sorting by price/quantity supports stock analysis.

4. Merge Sort ensures stable sorting performance.

5. Suitable for real-time retail systems with large datasets.

Task 5: Real-Time Stock Data Sorting & Searching

Scenario:

An AI-powered FinTech Lab at SR University is building a tool for analyzing

stock price movements. The requirement is to quickly sort stocks by daily

gain/loss and search for specific stock symbols efficiently.

• Use GitHub Copilot to fetch or simulate stock price data (Stock

Symbol, Opening Price, Closing Price).

• Implement sorting algorithms to rank stocks by percentage change.

• Implement a search function that retrieves stock data instantly when a

stock symbol is entered.

• Optimize sorting with Heap Sort and searching with Hash Maps.

• Compare performance with standard library functions (sorted(), dict

lookups) and analyze trade-offs.

Prompt:

Simulate stock data and implement Heap Sort for ranking by percentage change and Hash Map for searching by symbol.

Python Code:

```python
#Simulate stock data and implement Heap Sort for ranking by percentage change and Hash Map for searching by symbol.”
import heapq
class Stock:
    """
    A class to represent a stock with its symbol, price, and percentage change.
    """

    def __init__(self, symbol: str, price: float, percentage_change: float):
        self.symbol = symbol
        self.price = price
        self.percentage_change = percentage_change

    def __lt__(self, other):
        """Define less than for heap comparison based on percentage change."""
        return self.percentage_change < other.percentage_change
class StockMarket:
    """
    A class to represent a stock market that can store stocks and rank them by percentage change.
    It uses a heap to maintain the top stocks and a hash map for quick symbol lookup.
    """

    def __init__(self):
        self.stocks = []
        self.stock_map = {}

    def add_stock(self, stock: Stock):
        """Add a stock to the market."""
        heapq.heappush(self.stocks, stock)
        self.stock_map[stock.symbol] = stock

    def get_top_stocks(self, n: int) -> list:
        """Get the top n stocks by percentage change."""
        return heapq.nlargest(n, self.stocks)

    def search_stock(self, symbol: str) -> Stock:
        """Search for a stock by its symbol."""
```

```python
    def search_stock(self, symbol: str) -> Stock:
        """Search for a stock by its symbol."""
        return self.stock_map.get(symbol, None)
# Example usage:
if __name__ == "__main__":
    market = StockMarket()

    # Simulating stock data
    market.add_stock(Stock("AAPL", 150.00, 2.5))
    market.add_stock(Stock("GOOGL", 2800.00, -1.2))
    market.add_stock(Stock("AMZN", 3400.00, 3.0))
    market.add_stock(Stock("MSFT", 300.00, 1.5))

    # Displaying top stocks by percentage change
    print("Top Stocks by Percentage Change:")
    top_stocks = market.get_top_stocks(3)
    for stock in top_stocks:
        print(f"Symbol: {stock.symbol}, Price: {stock.price}, Percentage Change: {stock.percentage_change}%")

    # Searching for a stock by symbol
    symbol = "AAPL"
    stock = market.search_stock(symbol)
    if stock:
        print(f"\nSearch result for '{symbol}': Symbol: {stock.symbol}, Price: {stock.price}, Percentage Change: {stock.percentage_change}%")
    else:
        print(f"\nStock with symbol '{symbol}' not found.")
```

Output:

```
(.venv) PS C:\Users\DELL\OneDrive\Desktop\LEARNING COURSES\AI_Assisant_Coding> & "c:\Users\DELL\OneD
ive/Desktop/LEARNING COURSES/AI_Assisant_Coding/program-12.3.py"
Top Stocks by Percentage Change:
Symbol: AMZN, Price: 3400.0, Percentage Change: 3.0%
Symbol: AAPL, Price: 150.0, Percentage Change: 2.5%
Symbol: MSFT, Price: 300.0, Percentage Change: 1.5%

Search result for 'AAPL': Symbol: AAPL, Price: 150.0, Percentage Change: 2.5%
(.venv) PS C:\Users\DELL\OneDrive\Desktop\LEARNING COURSES\AI_Assisant_Coding>
```

Justification:

1. Heap Sort efficient for ranking top gainers/losers.
2. Hash Map provides instant stock symbol lookup O(1).
3. Handles frequent updates in stock data.
4. Comparison with built-in functions shows trade-offs.
5. Suitable for real-time financial data processing.